

REST 練習 2：一個 Service、多個資源——URI 路由

Lecture

rs01 示範的是「整個 Service 只有一個 Resource」的最簡單情況（Application 直接回傳一個 Resource 實例）。但實際的 API 通常有很多種資源（航班、訂位、乘客.....），每一種都需要自己的 URL 路徑跟自己的處理邏輯——這時候 Application 就不能只回傳單一 Resource，要有一個「路由器」依照 URL 路徑決定要交給哪一個 Resource。

CL_REST_ROUTER 就是這個角色：**ATTACH** 方法登記「URL 樣板 → Handler Class」的對應關係，一個 Router 可以掛很多條樣板。框架收到 request 後，Router 會依樣板比對 URL 路徑，找到對應的 Resource Class 並把 request 轉交給它。

為什麼要拆成 **Application**（總機）跟 **Resource**（工人）兩層，而不是一個 **Class** 全包：這是單一職責原則（Single Responsibility）的具體實踐——Application 只管「這個 request 該交給誰」，完全不碰業務邏輯；Resource 只管「收到 request 之後怎麼處理這一種資源」，完全不管路由。這樣的分工讓每個 Resource Class 可以獨立開發、獨立測試，也方便之後新增資源時只要多寫一個 Resource Class + 多一行 **ATTACH**，不用碰其他資源的程式碼。

這題也是本課程第一次出現「一個 Application 掛兩個 Resource」的寫法，之後 rs03~rs09 的 Application Class 幾乎都是這個模式（差別只在掛幾條路徑、掛哪些 Resource）。

學習目標

- 理解為什麼「一個 SICF Service 只能掛一個 Handler Class」，但實務上一個 Service 常常要處理很多種資源（/hello、/carriers、/flights.....）
- 會用 **CL_REST_ROUTER**：**ATTACH**(iv_template = 'URI樣板' iv_handler_class = '類別名稱字串') 把 URI Pattern 對應到 Resource Class
- 理解 router 是「收到 request 才動態 **CREATE OBJECT**」，**ATTACH** 只是註冊對照表，不會馬上建立 Resource 物件
- 能分辨「Application Class 決定路由規則」和「Resource Class 負責實際邏輯」這兩層職責不要混在一起
- （**延伸知識，非本題必考**）：理解如果這個 API 要給瀏覽器前端（React/Vue 等 SPA）直接呼叫，Application Class 除了路由還要多負責一件事——**CORS**（Cross-Origin Resource Sharing）。這跟 rs06 會教的 CSRF 是完全不同層級的兩個機制，容易被搞混，這裡先建立正確的心智模型，rs06 再對照兩者差異

為什麼需要 Router

rs01 的 **ZCL_RS01_APP** 直接 **ro_root_handler = NEW zcl_rs01_hello()**——這只在「整個 service 只有一種資源」時夠用。實務上一個 REST API 通常要同時提供好幾種資源（例如訂單、客戶、航班.....），每種資源各自有一個 Resource Class，但 SICF 一個 Service 只能指定一個 Handler Class（Application Class）。解法：Application Class 不直接處理，而是回傳一個 **CL_REST_ROUTER** 實例，router 依照 URL 路徑決定要交給哪個 Resource Class。

CORS：如果要給瀏覽器前端呼叫，還要多做一件事（**延伸知識**）

前面九題的測試工具都是 `SPROX_HTTP_REQUEST` (SAP GUI 內部程式) 或 Postman/curl——這些工具不會觸發瀏覽器的同源政策 (Same-Origin Policy)，所以這門課到目前為止完全感覺不到 CORS 的存在。但如果呼叫端換成一支跑在瀏覽器裡的網頁 (例如部署在 `https://frontend.example.com` 的 React App，要呼叫 `https://sap.company.com/sap/bc/zrest_training/...`)，瀏覽器會主動擋下「跨來源」的回應，除非伺服器明確用 HTTP header 表態「我允許這個來源讀取回應」。

這是瀏覽器自己的安全機制，不是 SAP 或 ABAP 的行為——伺服器其實還是正常處理了請求、也正常回了 200，只是瀏覽器收到回應後，發現裡面沒有 `Access-Control-Allow-Origin` 這個 header，就不讓呼叫的 JavaScript 讀取這個回應內容 (在瀏覽器的 Network 頁籤看得到請求成功，但 Console 會報一個 CORS 錯誤)。

Preflight (預檢請求)：如果前端呼叫用了 `Content-Type: application/json` 或帶了自訂 Header (例如 rs06 會教的 `X-CSRF-Token`)，瀏覽器在送出真正的 GET/POST 之前，會先自動送一個 **OPTIONS** 請求探路，帶著 `Access-Control-Request-Method/Access-Control-Request-Headers` 問伺服器「我等一下要用這個方法、帶這些 header，你允許嗎？」。伺服器要回：

```
Access-Control-Allow-Origin: https://frontend.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, X-CSRF-Token
```

瀏覽器確認允許後，才會真的送出正式請求；正式請求的回應也一樣要帶 `Access-Control-Allow-Origin`，瀏覽器才會放行讓 JS 讀到內容。

`CL_REST_HTTP_HANDLER` 沒有像 **CSRF** 那樣內建處理 **CORS**，要自己刻，而且要放在 **Application Class**、覆寫 `IF_HTTP_EXTENSION~HANDLE_REQUEST` (這個方法比 `GET_ROOT_HANDLER` 更早被呼叫，是整個框架的最外層入口)，理由：

1. 這一層可以涵蓋這個 Service 底下所有 Resource、所有動詞的回應，寫在個別 Resource 裡每支都要補一次，容易漏
2. **Preflight** 的 **OPTIONS** 請求瀏覽器不會帶認證資訊 (沒有帳密、沒有已登入的 session)，如果照一般流程跑到 Resource 才處理，**OPTIONS** 會先被 Basic Auth 擋下要求登入而失敗——**preflight** 一失敗，瀏覽器根本不會送出後面的正式請求，所以 **OPTIONS** 必須在最外層就攔截處理，完全跳過認證與路由

如何只允許白名單裡的網址 (而不是開放任何來源)：

```
CLASS zcl_rs02_app DEFINITION INHERITING FROM cl_rest_http_handler.
  PROTECTED SECTION.
  METHODS if_http_extension~handle_request REDEFINITION.
ENDCLASS.

CLASS zcl_rs02_app IMPLEMENTATION.

  METHOD if_http_extension~handle_request.
    " 白名單：只有這幾個網址可以跨來源呼叫本 API
    DATA(lt_allowed_origins) = VALUE string_table(
      ( `https://frontend.example.com` )
      ( `http://localhost:3000` ) " 本機開發用
    )
  ENDMETHOD.
ENDCLASS.
```

```

    ).

    DATA(lv_origin) = server->request->get_header_field( iv_name = 'origin' ).

    IF lv_origin IS NOT INITIAL AND line_exists( lt_allowed_origins[ table_line =
lv_origin ] ).
        " 注意：回填的是「比對到的 lv_origin 本身」，不是寫死一個固定網址、也不是 '*'
        server->response->set_header_field( iv_name = 'Access-Control-Allow-Origin'
iv_value = lv_origin ).
        server->response->set_header_field( iv_name = 'Access-Control-Allow-Methods'
iv_value = 'GET, POST, PUT, DELETE, OPTIONS' ).
        server->response->set_header_field( iv_name = 'Access-Control-Allow-Headers'
iv_value = 'Content-Type, X-CSRF-Token' ).
        server->response->set_header_field( iv_name = 'Access-Control-Max-Age'
iv_value = '3600' ).
    ENDIF.

    IF server->request->get_method( ) = 'OPTIONS'.
        " Preflight 到此結束：只回 CORS header，不呼叫 SUPER->，不進路由、不驗證 CSRF/帳密
        server->response->set_status( code = 200 reason = 'OK' ).
        RETURN.
    ENDIF.

    " 非 OPTIONS 的正式請求：交還框架繼續走 CSRF 檢查 / 路由 / Resource 那一整套流程
    super->if_http_extension~handle_request( server ).
ENDMETHOD.

ENDCLASS.

```

幾個容易誤解的地方：

- ****Access-Control-Allow-Origin** 回填的值必須是「比對白名單後、原封不動的 `lv_origin`」，不能圖方便寫死 *******：* 代表「任何來源都允許」，一來不安全（等於沒做白名單管控），二來瀏覽器規定 * 不能跟需要帶 Cookie/認證資訊的請求（`credentials: 'include'`）並用——只要前端 fetch 有帶 `credentials`，伺服器就一定要回填「精確比對到的來源」而不是 *，這也是上面範例先查白名單、再回填 `lv_origin` 本身的原因
- 這段程式碼放的位置是 **IF_HTTP_EXTENSION~HANDLE_REQUEST**，不是 **IF_REST_APPLICATION~GET_ROOT_HANDLER**——**GET_ROOT_HANDLER** 只負責回傳 router，不會被呼叫在「連 CSRF/路由都還沒開始跑」的最外層時間點；**HANDLE_REQUEST** 才是整個 **CL_REST_HTTP_HANDLER** 框架的入口，這也是 rs01 團隊實務備註提過「**GET_ROOT_HANDLER** 是唯一要覆寫的方法」在這個延伸情境下的例外
- 這門課用的測試工具測不出 **CORS 問題**：**SPROX_HTTP_REQUEST**、curl、Postman 都不是瀏覽器，不會執行同源政策檢查，就算沒加任何 CORS header 它們一樣能正常呼叫、正常拿到回應內容——CORS 只有真正的瀏覽器（或瀏覽器裡跑的 JS）才會擋。想驗證伺服器有沒有回對 header，可以用 `curl -i -H "Origin: https://frontend.example.com" http://<主機>:<port>/sap/bc/zrest_training/rs02/carriers` 看 Response Header 裡有沒有 **Access-Control-**

`Allow-Origin`，但這只能驗證「伺服器行為對不對」，驗證不了「瀏覽器真的會放行」——真要驗證後者，得有一支部署在不同來源的網頁實際用 `fetch()` 呼叫

事前準備

ADT 端物件已由課程準備好 (`$TMP`)：

- `ZCL_RS02_APP`——Application Class，改用 `router`
- `ZCL_RS02_HELLO`——沿用 `rs01` 的問候邏輯
- `ZCL_RS02_CARRIERS`——查 `SCARR` 回傳純文字清單 (JSON 序列化留到 `rs03`)

題目需求 (對照已建好的答案物件)

1. `ZCL_RS02_APP` 的 `GET_ROOT_HANDLER`：

```
`abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template = '/hello'
iv_handler_class = 'ZCL_RS02_HELLO' ). lo_router->attach( iv_template = '/carriers'
iv_handler_class = 'ZCL_RS02_CARRIERS' ). ro_root_handler = lo_router.` 注意
iv_handler_class 是字串，不是物件參考—router 內部用 CREATE OBJECT ... TYPE (class_name) 動態建立，
兩個 Resource Class 之間、甚至跟 router 之間都不需要互相 import`
```

1. `ZCL_RS02_HELLO`：跟 `rs01` 的 `ZCL_RS01_HELLO` 邏輯相同 (只覆寫 `GET`，回傳純文字問候)
2. `ZCL_RS02_CARRIERS`：覆寫 `GET`，`SELECT carrid, carrname FROM scarr ORDER BY carrid INTO TABLE @DATA(lt_carrier)`，用 `REDUCE string( )` 組成每行一筆的純文字清單回傳

SICF 掛載步驟 (比照 `rs01`，只是換掉數值)

沿用 `rs01` 已建好的 `/sap/bc/zrest_training` 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 `rs02`
2. Handler List 掛 `ZCL_RS02_APP` (不是 `ZCL_RS01_APP`)
3. Activate Service
4. 測試兩個路徑：
 - `http://<主機>:<port>/sap/bc/zrest_training/rs02/hello?sap-client=130`
 - `http://<主機>:<port>/sap/bc/zrest_training/rs02/carriers?sap-client=130`

預期輸出 (範例)

`/hello`：

```
Hello REST! 現在伺服器時間是 14:40:12
```

`/carriers`：

```
AA  American Airlines
LH  Lufthansa
UA  United Airlines
...
```

團隊實務備註

- **ATTACH** 的 URI 樣板語法支援 {變數} (如 `/carriers/{carrid}`) 擷取路徑參數，但這題先只用固定路徑，路徑參數留到 rs04
- Router 用 **CREATE OBJECT TYPE** (字串) 動態建立物件，代表：**Resource Class 名稱一旦打錯字，編譯期不會抓到，要實際呼叫那個 URL 才會發現** (通常是 404 或 dump) ——這是動態建立物件的通病，op08 學過的多型轉型是靜態繫結，這裡是完全不同的動態機制
- 一個 Application Class 可以掛任意多個 **ATTACH**，實務上大型 API 會用同一個 router 管理十幾個資源

思考題

1. 如果 `/carriers` 和 `/carrier` 兩個路徑都 **ATTACH** 了不同的 Resource Class，**CL_REST_ROUTER** 怎麼判斷用哪一個？ (提示：看 **FIND_MATCH / GET_FIRST_MATCH** 的排序邏輯——精確字面比對的優先權高於帶變數的樣板)
2. **ZCL_RS02_CARRIERS** 現在回傳純文字、用 `\t` 分欄——如果呼叫端是一支前端 JS 程式，這種格式好解析嗎？ (這正是 rs03 要解決的問題)
3. 承 op05「全域類別」：**ZCL_RS02_HELLO**、**ZCL_RS02_CARRIERS** 這兩個 Resource Class 彼此完全不知道對方存在，也不需要共同的父類別或介面——router 是怎麼做到「不用介面也能一致呼叫」的？

答案

見 `zcl_rs02_app.clas.abap`、`zcl_rs02_hello.clas.abap`、`zcl_rs02_carriers.clas.abap` (SAP 端物件 **ZCL_RS02_APP / ZCL_RS02_HELLO / ZCL_RS02_CARRIERS**)。SICF Service 路徑 `/sap/bc/zrest_training/rs02`。